Operating Systems

GATE CSE 2027 · complete formula, rule, method & trap sheet

135 reference boxes · **9** chapters ·

System Calls, Processes & Threads · CPU Scheduling · Synchronization & IPC · Deadlocks · Memory Management & Paging · Virtual Memory · File Systems, Disk Scheduling & I/O · Problem → Standard Method Playbook · 🎯 Representative GATE-Style Worked Problems · Last-minute one-liners

1 System Calls, Processes & Threads

What is an OS? High-yield

NOTES

Think of the OS as the **manager of a shared house**: the hardware (CPU, RAM, disk) is the house, programs are the tenants, and the OS decides who gets the kitchen (CPU) and when, keeps tenants out of each other's rooms (memory protection), and gives everyone simple keys instead of raw wiring (abstraction: files instead of disk sectors).

KEY POINT

User mode

Normal programs run here. Cannot touch hardware or other programs' memory directly.

KEY POINT

Kernel mode

The OS core runs here with full power. A mode bit in the CPU tracks which mode you are in.

System calls Core

NOTES

A **system call** is how a user program politely asks the kernel to do something privileged (open a file, create a process, allocate memory). It works via a **trap**: a special instruction that flips the CPU into kernel mode and jumps to a fixed kernel routine.

FLOW

User program calls read() → TRAP → mode bit = kernel → Kernel runs the service → Return → mode bit = user

NOTES

Common ones: `fork` (create process), `exec` (replace process image), `wait`, `exit`, `open/read/write/close`.

GATE FAVOURITE

GATE trap

Privileged instructions (I/O, halt, changing the mode bit, memory-management registers) work only in kernel mode. Trying them in user mode raises a trap. Also: a **trap/system call is a software (synchronous) event**, an **interrupt is a hardware (asynchronous) event**.

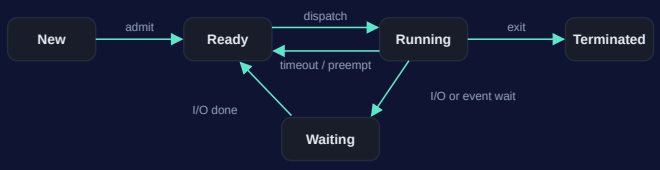
Process = a program in execution Core

NOTES

A program is a file sitting on disk. A **process** is that program running: it has its own address space with **text** (code), **data** (globals), **heap** (malloc) and **stack** (function calls), plus a record the OS keeps about it called the **PCB** (Process Control Block): PID, state, program counter, CPU registers, memory info, open files, scheduling info.

Process state diagram

DIAGRAM



Context switch

NOTES

When the CPU moves from process A to B, the OS **saves A's registers/PC into A's PCB and loads B's**. It's pure overhead (no user work happens), so the OS wants it cheap and rare.

fork() — the favourite GATE question Core

NOTES

`fork()` clones the calling process. Both continue from the next line. It returns **0 in the child**, the **child's PID in the parent**, and −1 on failure. Each fork **doubles** the number of processes.

WORKED EXAMPLE

Worked numerical

Q: How many processes exist (including the original) after `fork(); fork(); fork();` and how many times is `printf("hi")` printed if it comes after all three?

A: 1 → 2 → 4 → 8 processes, so total = $2^n = 2^3 =$ **8** (7 children). Every process runs the printf → **8 times**.

WORKED EXAMPLE

Worked numerical

Q: How many *child* processes are created by `if (fork() == 0) fork();` ?

A: Parent forks child C1 (in C1, fork returned 0). Only C1 enters the if-body and forks C2. The parent skips it. Total = 3 processes → **2 children** (C1, C2). Rule: **a fork inside a condition only doubles the processes that satisfy the condition**.

WORKED EXAMPLE

Worked numerical

Q: `for (i = 0; i < 4; i++) fork();` — how many processes in total?

A: Every process (including new ones) executes every remaining iteration, so it doubles each round: $2^4 =$ **16** total, **15** newly created.

Threads Core

NOTES

A **thread** is a lightweight unit of execution *inside* a process. Threads of one process **share** code, data, heap, open files; each has its **own** stack, registers and PC. Creating a thread and switching between threads is much cheaper than for processes.

REFERENCE TABLE

	User-level threads (ULT)	Kernel-level threads (KLT)
Managed by	Library in user space, kernel unaware	The kernel itself
Switch cost	Very low, no trap needed	Higher (needs kernel entry)
Blocking syscall	Blocks the **whole process**	Only that thread blocks
Multi-core parallelism	No (kernel sees 1 thread)	Yes

COMMON TRAP

Remember

Threads share the heap and globals, so they need synchronization (Part 2). Each thread keeps its **own stack and register set**, that is why local variables are not shared.

Types of schedulers & the dispatcher High-yield

REFERENCE TABLE

Scheduler	Job	How often it runs
Long-term (job scheduler)	Admits jobs from the job pool into memory; controls the **degree of multiprogramming** and the mix of CPU-bound and I/O-bound jobs	Rarely (seconds to minutes)
Short-term (CPU scheduler)	Picks which ready process gets the CPU next	Very often (every few ms) — must be fast
Medium-term (swapper)	Swaps processes out of and back into memory (the suspended states) to lower the degree of multiprogramming temporarily	Occasionally

MUST REMEMBER

Dispatcher = the module that actually hands the CPU to the process the short-term scheduler chose: it performs the **context switch**, switches to user mode and jumps to the right place in the user program. The time it takes is **dispatch latency**. Remember: the *scheduler decides*, the *dispatcher acts*.

Multithreading models Extension

REFERENCE TABLE

Model	Mapping	Consequences
Many-to-one	Many user threads → 1 kernel thread	Cheap, but one blocking system call blocks the whole process, and there is no multicore parallelism
One-to-one	Each user thread → its own kernel thread	True parallelism, no blocking of siblings; costs a kernel thread per user thread (Linux, Windows)
Many-to-many	n user threads multiplexed onto m ≤ n kernel threads	Flexible and parallel, but harder to implement (the two-level model also allows binding a user thread to one kernel thread)

Zombie and orphan processes High-yield

KEY POINT

Zombie (defunct)
The process has **terminated**, but its process-table entry (PID and exit status) stays because the parent has not yet called `wait()`. It uses no CPU or memory, only a table slot. Removed when the parent calls `wait()`, or when the parent dies and `init` reaps it.

KEY POINT

Orphan
The process is **still running** but its **parent has terminated first**. It is adopted by `init` (PID 1, or systemd), which later calls `wait()` for it, so orphans never become permanent zombies.

GATE FAVOURITE

GATE trap
Zombie = dead but not reaped. Orphan = alive but parentless. You cannot "kill" a zombie — it is already dead; the fix is to make its parent reap it.

Copy-on-write (COW) fork High-yield

KEY POINT

A naive `fork()` copies the whole address space, which is wasted if the child calls `exec()` immediately. With COW the child initially **shares the parent's physical frames**, all marked read-only. The first **write** to a shared page raises a protection fault; the OS then copies *only that page* and makes the copy private and writable. Pages that are never written are never copied.

WORKED EXAMPLE

Worked numerical
Q: A parent with 100 pages forks a child. The child writes to 10 pages and the parent writes to none. How many extra frames are used, with and without COW?
A: Without COW: 100 extra frames (full copy). With COW: only the 10 written pages are copied → **10 extra frames**, and the other 90 stay shared.

2 CPU Scheduling

Terms you must know Core

REFERENCE TABLE

Term	Meaning
Arrival Time (AT)	When the process enters the ready queue
Burst Time (BT)	CPU time it needs
Completion Time (CT)	When it finishes
Turnaround Time (TAT)	`CT − AT`
Waiting Time (WT)	`TAT − BT` (time spent waiting in ready queue)
Response Time (RT)	First time on CPU − AT

NOTES

Preemptive = the OS may take the CPU away from a running process (on timeout or when a better process arrives). **Non-preemptive** = process keeps the CPU until it finishes or blocks.

The algorithms Core

REFERENCE TABLE

Algorithm	Rule	Type	Watch out
FCFS	First come, first served	Non-preemptive	Convoy effect: one long job delays everyone
SJF	Shortest burst first	Non-preemptive	Minimum avg WT; long jobs may starve
SRTF	Shortest *remaining* time first	Preemptive SJF	Re-check at every arrival
Round Robin	Each gets a time quantum q, then goes to back of queue	Preemptive	Big q → FCFS, tiny q → all overhead
Priority	Highest priority first	Either	Starvation; fixed by **aging** (raise priority over time)

Worked numerical 1: FCFS vs SRTF Core

REFERENCE TABLE

Process	AT	BT
P1	0	8
P2	1	4
P3	2	9
P4	3	5

GANTT CHART

FCFS Gantt chart
P1 [0–8] · P2 [8–12] · P3 [12–21] · P4 [21–26]

FORMULA / WORKING

```
CT:  P1=8  P2=12  P3=21  P4=26
TAT = CT-AT: 8, 11, 19, 23   → avg 15.25
WT  = TAT-BT: 0, 7, 10, 18   → avg WT = 35/4 = 8.75
```

GANTT CHART

SRTF Gantt chart (P2 preempts P1 at t=1; at t=5 shortest remaining is P4=5, then P1=7, then P3=9)

P1 [0–1] · P2 [1–5] · P4 [5–10] · P1 [10–17] · P3 [17–26]

FORMULA / WORKING

```
CT:  P1=17  P2=5  P3=26  P4=10
TAT: 17, 4, 24, 7
WT : 9, 0, 15, 2   → avg WT = 26/4 = 6.5
```

NOTES

SRTF beat FCFS (6.5 vs 8.75). That is the whole point of favouring short jobs.

Worked numerical 2: Round Robin Core

NOTES

P1 BT=24, P2 BT=3, P3 BT=3, all arrive at 0, quantum q = 4.

GANTT CHART

P1 [0–4] · P2 [4–7] · P3 [7–10] · P1 [10–14] · P1 [14–18] · P1 [18–22] · P1 [22–26] · P1 [26–30]

FORMULA / WORKING

```
CT: P1=30  P2=7  P3=10
WT = CT - AT - BT:  P1=30-24=6,  P2=7-3=4,  P3=10-3=7
avg WT = 17/3 ≈ 5.67
```

GATE FAVOURITE

GATE convention

In Round Robin, if a new process arrives at the same instant a running process is preempted, the **new arrival goes into the ready queue first**, then the preempted process. Read the question: if it states a different rule, follow it.

Context-switch overhead High-yield

WORKED EXAMPLE

Worked numerical

Q: Round Robin with quantum q = 10 ms and context-switch time s = 2 ms. What fraction of time is useful work (assume every quantum is fully used)?
A: Each cycle = q + s = 12 ms, of which 10 ms is useful → efficiency = 10/12 = **83.3%** (overhead 16.7%). General: `efficiency = q / (q + s)`.

QUICK FACTS

Quick Facts for Exam

- Among non-preemptive algorithms, SJF gives the **minimum average waiting time**; SRTF is optimal among preemptive ones.
- FCFS has the convoy effect; RR gives the best (lowest) **response time** for interactive systems.
- I/O-bound processes use short CPU bursts; CPU-bound use long bursts. Good schedulers favour I/O-bound ones to keep devices busy.
- If all processes arrive at t=0, FCFS and non-preemptive SJF need no arrival-time juggling, just sort by order/burst.

Highest Response Ratio Next (HRRN) High-yield

KEY POINT

```
Response ratio   RR = (W + S) / S  =  1 + W/S
W = waiting time so far,   S = service (burst) time
```

Non-preemptive. At every scheduling point, run the ready process with the **highest RR**. For the same waiting time a short job has a higher RR (small S), while a long job's RR keeps rising the longer it waits — so HRRN favours short jobs **without starving long ones**. It needs burst-time estimates, like SJF.

REFERENCE TABLE

Algorithm	Starvation?	Weak point
FCFS	No	Convoy effect
SJF	Yes (long jobs)	Needs burst times; unfair to long jobs
HRRN	No	Needs burst times; RR must be recomputed at each decision

WORKED EXAMPLE

Worked numerical

Q: P1 (arrives 0, burst 3), P2 (1, 6), P3 (2, 2). Schedule with non-preemptive HRRN.
A:

```
t = 0 : only P1 is ready → runs 0–3.
t = 3 : P2 has W = 2, S = 6 → RR = (2+6)/6 = 1.33
        P3 has W = 1, S = 2 → RR = (1+2)/2 = 1.50   → P3 runs 3–5.
t = 5 : only P2 is left (RR = (4+6)/6 = 1.67) → runs 5–11.
Waiting times: P1 = 0, P3 = 1, P2 = 4 → average = 5/3 ≈ 1.67 (FCFS gives 3.0)
```

Multilevel queue vs multilevel feedback queue (MLFQ) High-yield

REFERENCE TABLE

	Multilevel queue	Multilevel feedback queue
Queue assignment	Fixed by process type (system, interactive, batch)	Dynamic — jobs **move between queues** by observed behaviour
Per-queue policy	Each queue may use its own algorithm (e.g. RR for interactive, FCFS for batch)	Same idea: top queues small quantum RR, bottom queue FCFS
Between queues	Fixed priority, or time slices shared among queues	Strict priority; lower queues run only when higher ones are empty
Starvation	Possible for low queues	Prevented by **aging** (promote jobs that wait too long)

KEY POINT

MLFQ rules: a new job enters the **top** queue (highest priority, smallest quantum). If it uses its **whole quantum** it is demoted one level (it looks CPU-bound); if it blocks for I/O before the quantum ends it stays high (it looks interactive). So MLFQ favours short and I/O-bound jobs **without knowing burst lengths in advance** — it approximates SRTF. Design parameters: number of queues, algorithm per queue, promotion rule, demotion rule, and the queue a new job enters.

WORKED EXAMPLE

Worked numerical

Q: Q0 = RR (quantum 4), Q1 = RR (quantum 8), Q2 = FCFS. P1 (burst 20), P2 (6) and P3 (3) all arrive at 0 in that order. Find the completion times.
A:

```
Q0 : P1 0–4 (16 left → Q1)   P2 4–8 (2 left → Q1)   P3 8–11 done at 11
Q1 : P1 11–19 (8 left → Q2)  P2 19–21  done at 21
Q2 : P1 21–29  done at 29
Completion: P3 = 11, P2 = 21, P1 = 29.   Waiting (= CT − BT): P1 9, P2 15, P3 8 → average 32/3 ≈ 10.67
```

The shortest job (P3) finishes first and the longest (P1) sinks to the bottom queue — SRTF-like behaviour with no advance knowledge.

3 Synchronization & IPC

Race condition & the critical section Core

NOTES

If two threads update shared data at the same time, the final answer depends on **who wins the race**. Even `x = x + 1` is really three steps: *load x → add 1 → store x*, and a context switch can land between them. The code touching shared data is the **critical section (CS)**.

FLOW

Entry section → Critical section → Exit section → Remainder

KEY POINT

A correct solution needs all three:

1. **Mutual exclusion**: only one process in the CS at a time.
2. **Progress**: if the CS is free, one of the waiting processes must be allowed in (no needless blocking by processes outside the CS).
3. **Bounded waiting**: a limit on how many times others can enter before a waiting process gets its turn (no starvation).

Peterson's solution (2 processes, pure software) Core

FORMULA / WORKING

```
// shared: flag[2] = {false,false};  int turn;
// process i (j = 1-i):
flag[i] = true;          // I want to enter
turn = j;                // but you go first if you also
want
while (flag[j] && turn == j);   // busy wait
   /* critical section */
flag[i] = false;         // exit
```

NOTES

It satisfies mutual exclusion, progress **and** bounded waiting. Real hardware uses atomic instructions like `test-and-set` / `compare-and-swap` instead. Note: a plain lock variable (`while(lock); lock=1;`) does **not** work, because checking and setting are two separate steps.

Semaphores Core

NOTES

A semaphore is an integer `S` touched only by two **atomic** operations:

FORMULA / WORKING

```
wait(S)   /* P */ :  S--;  if (S < 0) block this process
signal(S) /* V */ :  S++;  if (S <= 0) wake up one blocked
process
```

KEY POINT

Binary semaphore / mutex

Value 0 or 1. Used for mutual exclusion: init to 1, `wait` before the CS, `signal` after.

KEY POINT

Counting semaphore

Counts available units of a resource (e.g. 5 printers, or free buffer slots).

WORKED EXAMPLE

Worked numerical

Q: A counting semaphore starts at S = 3. After 5 wait() and 1 signal() operations, what is S and how many processes are blocked?
A: S = 3 − 5 + 1 = **−1**. A negative value means |S| processes are blocked → **1 process blocked**. (Rule: S ≥ 0 ⇒ S free units; S < 0 ⇒ |S| waiting processes.)

WORKED EXAMPLE

Worked numerical

Q: Semaphore S = 10 initially. 6 wait() and 4 signal() operations complete. What is S?
A: 10 − 6 + 4 = **8** (nobody blocked, since S never went below 0).

Classic problems Core

1. Producer–Consumer (bounded buffer of N slots)

FORMULA / WORKING

```
semaphore mutex = 1, empty = N, full = 0;

Producer:                Consumer:
 wait(empty);             wait(full);
 wait(mutex);             wait(mutex);
   add item                 remove item
 signal(mutex);           signal(mutex);
 signal(full);            signal(empty);
```

NOTES

`empty` counts free slots, `full` counts filled slots, `mutex` guards the buffer.

GATE FAVOURITE

GATE trap

Swap the order to `wait(mutex); wait(empty);` in the producer and you get a **deadlock**: a producer holding the mutex sleeps on a full buffer, and the consumer can never take the mutex to free a slot. The order of waits matters, the order of signals does not.

2. Readers–Writers

NOTES

Many readers may read together, but a writer needs exclusive access. The simple solution favours readers (writers can starve); a writer-priority version flips this. Uses a `read_count` protected by a mutex, and a `wrt` semaphore for writers.

3. Dining Philosophers

NOTES

5 philosophers, 5 forks, each needs both neighbouring forks. If everyone grabs their left fork at the same time, all wait forever → **deadlock**. Fixes: allow at most 4 to sit; pick up both forks atomically; or make one philosopher pick right-first (breaks circular wait).

NOTE

GATE classic: lost updates

Q: Two threads each run `for (i=0;i<N;i++) x = x + 1;` on a shared `x = 0`, with no synchronization and non-atomic load/add/store. What are the maximum and minimum possible final values of x (N ≥ 2)?
A: Max = 2N (no interleaving damage). **Min = 2**:
1. T1 loads x = 0, then is paused.
2. T2 runs N−1 full iterations → x = N−1.
3. T1 stores 1. T2 loads 1 for its last iteration and is paused.
4. T1 runs its remaining N−1 iterations → x = N.
5. T2 stores 1+1 = **2**. Final x = 2.

Hardware support: test-and-set, swap and compare-and-swap Core

NOTES

Pure-software locks (Peterson) are rarely used on real machines. The CPU instead gives **atomic read-modify-write** instructions that cannot be interrupted halfway:

FORMULA / WORKING

```
boolean TestAndSet(boolean *lock) {      // executes atomically
    boolean old = *lock;  *lock = true;  return old;
}
// shared: boolean lock = false;
do {
    while (TestAndSet(&lock)) ;        // spin until old value was false
        /* critical section */
    lock = false;
} while (true);

int CompareAndSwap(int *v, int expected, int new_value) {   // atomic
    int tmp = *v;
    if (tmp == expected) *v = new_value;
    return tmp;
}
```

REFERENCE TABLE

Property	Simple test-and-set lock
Mutual exclusion	**Yes** — only the process that saw `false` enters
Progress	**Yes**
Bounded waiting	**No** — a fast process can release and re-grab the lock every time, so another one can spin forever (starvation). A bounded-waiting version adds a `waiting[n]` array and hands the lock to the next waiting process in cyclic order
Cost	Busy waiting: the CPU burns cycles while spinning

MUST REMEMBER

Spinlock = a lock built on busy waiting. It wastes CPU on a single core (the lock holder cannot run while you spin), but on a **multiprocessor** it beats blocking when the critical section is shorter than a context switch. A **blocking mutex/semaphore** instead puts the waiter to sleep and wakes it on release. Rule of thumb: **short CS on many CPUs → spinlock; long CS → sleep.**

GATE FAVOURITE

GATE trap

Disabling interrupts also gives mutual exclusion, but only on a **single processor** and only for kernel code — on a multiprocessor the other CPUs keep running, so it does not work. And a plain `while(lock); lock=1;` fails because checking and setting are two separate steps; test-and-set fuses them into one atomic step.

Monitors & condition variables High-yield

NOTES

A **monitor** is a language-level construct: shared data plus the procedures that touch it, packaged so that **only one process can be active inside the monitor at a time**. The compiler/runtime supplies the mutual exclusion, so the programmer cannot forget a `wait(mutex)`. To wait for a *condition* (e.g. "buffer not empty") the monitor uses **condition variables** with two operations:

FORMULA / WORKING

```
x.wait()    // the caller releases the monitor lock and blocks on x
x.signal()  // wake ONE process waiting on x; if none is waiting, it does NOTHING

monitor BoundedBuffer {
    int count = 0;  condition notFull, notEmpty;
    void put(item) {
        while (count == N) notFull.wait();   // while, not if (Mesa-safe)
        add item; count++;
        notEmpty.signal();
    }
    item get() {
        while (count == 0) notEmpty.wait();
        remove item; count--;
        notFull.signal();
        return item;
    }
}
```

REFERENCE TABLE

Semaphore	Condition variable in a monitor
`signal(S)` is **remembered** (S is incremented even if nobody waits)	`x.signal()` is **lost** if no process is waiting on x
`wait(S)` blocks only if S would go negative	`x.wait()` **always** blocks
Programmer must pair every wait/signal correctly	Mutual exclusion is automatic; only the condition logic is manual

KEY POINT

Signal semantics. When P calls `x.signal()` and Q is waiting on x, both cannot run inside the monitor at once. **Hoare (signal-and-wait):** P hands the monitor to Q immediately and waits, so Q knows the condition is still true — an `if` test is enough. **Mesa (signal-and-continue):** P keeps running and Q only becomes ready; the condition may have changed before Q gets in, so Q must **re-check in a `while` loop**. Most real systems (Java, pthreads) use Mesa semantics, so always write `while (cond) x.wait();`.

Priority inversion & priority inheritance High-yield

NOTES

Priority inversion happens when a **high-priority** task is stuck waiting for a lock held by a **low-priority** task, and a **medium-priority** task that does not need the lock keeps preempting the low one — so the high-priority task is effectively delayed by the medium one.

FORMULA / WORKING

```
L (low) takes lock → H (high) arrives, blocks on the lock
→ M (medium) arrives and preempts L (M has higher priority than L)
→ L cannot run, so it cannot release the lock → H waits as long as M runs
```

KEY POINT

Fix — priority inheritance: while L holds a lock that H wants, L temporarily **inherits H's priority**. M can no longer preempt L, L finishes its critical section quickly, releases the lock, drops back to low priority, and H runs. (The 1997 Mars Pathfinder resets are the textbook real-world example.) A related fix is the **priority ceiling** protocol: a lock's ceiling is the highest priority of any task that may use it, and a task holding it runs at that ceiling.

WORKED EXAMPLE

Worked example

Q: Priorities H > M > L. L starts at t=0 and locks R; it needs 4 ms of CPU inside the lock in total. H arrives at t=2 and requests R (it blocks). M arrives at t=3 and needs 10 ms of CPU. When does H get R (a) without and (b) with priority inheritance?

A: By t=2 L has used 2 ms in the lock. **(a) Without inheritance:** L runs t=2→3 (1 ms more, 1 ms left). At t=3 M preempts L, since M outranks L, and runs t=3→13. L finishes its last 1 ms at t=13→14, so **H gets R at t=14** — held up by M's entire burst although H outranks M.

(b) With inheritance: from t=2 L runs at H's priority, so M cannot preempt it. L runs t=2→4, releases R, and **H gets R at t=4**.

Deadlock vs livelock vs starvation High-yield

REFERENCE TABLE

	Deadlock	Livelock	Starvation
State of processes	Blocked, waiting for each other	**Running** and changing state, but making no progress	Ready/waiting, but never chosen
Typical cause	Circular wait on resources	Both keep reacting to each other (two people sidestepping in a corridor; repeated retry/back-off in lock-step)	Unfair scheduling or lock policy (SJF, reader-priority)
Fix	Prevent, avoid, detect and recover	Randomised back-off, break the symmetry	Aging, fair queues

Inter-Process Communication (IPC) Core

REFERENCE TABLE

Method	Idea
Shared memory	Fastest: processes map the same region, but you must synchronize yourself
Message passing	send/receive through the kernel; easier and safer, slower
Pipes	One-way byte stream (parent-child); named pipes (FIFO) work between unrelated processes
Sockets	Two-way, works across machines

4 Deadlocks

The four necessary conditions (Coffman) Core

NOTES

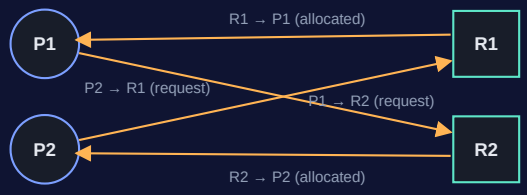
A deadlock can occur only if **all four** hold at once:

REFERENCE TABLE

Condition	Meaning	Break it by
Mutual exclusion	A resource is held by one process at a time	Make resources sharable (rarely possible)
Hold and wait	Holds some resources while waiting for more	Request everything at once
No preemption	Resources cannot be forcibly taken away	Allow the OS to preempt
Circular wait	P1 waits for P2 waits for ... waits for P1	Number resources, request in increasing order

Resource Allocation Graph (RAG) Core

DIAGRAM



NOTES

Edge **R → P** = allocated, **P → R** = requested. Here P1 holds R1 and wants R2, while P2 holds R2 and wants R1: a cycle.

COMMON TRAP

GATE rule

- Every resource has **a single instance**: cycle ⇔ deadlock.
- Resources have **multiple instances**: cycle is necessary but **not sufficient**. Check whether a free instance or a process outside the cycle can still finish.
- No cycle ⇒ no deadlock.

Handling deadlocks Core

KEY POINT

Prevention
Design so at least one Coffman condition can never hold.

KEY POINT

Avoidance
Grant a request only if the system stays in a **safe state** (Banker's algorithm).

KEY POINT

Detection & recovery
Let deadlock happen, detect via graph/algorithm, then kill processes or preempt resources.

KEY POINT

Ignore (ostrich)
What Linux/Windows mostly do: deadlocks are rare, so avoid the overhead.

NOTES

Safe state = there exists an order (safe sequence) in which every process can finish. Safe ⇒ no deadlock; unsafe ⇒ deadlock *possible* (not certain).

Banker's algorithm Core

NOTES

Keep `Need = Max − Allocation`. Repeatedly pick any process whose Need ≤ Available, let it finish, and add its Allocation back to Available. If all can finish, the state is safe.

WORKED EXAMPLE

Worked numerical

Q: 5 processes, resources A, B, C. Available = (3,3,2). Is the state safe?

Proc	Allocation	Max	Need = Max − Alloc
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

A: Work = Available (3,3,2).

```
P0 need 7 4 3 > 3 3 2  → cannot run yet
P1 need 1 2 2 ≤ 3 3 2  → run. Work = 3 3 2 + 2 0 0 = 5 3 2
P3 need 0 1 1 ≤ 5 3 2  → run. Work = 5 3 2 + 2 1 1 = 7 4 3
P4 need 4 3 1 ≤ 7 4 3  → run. Work = 7 4 3 + 0 0 2 = 7 4 5
P2 need 6 0 0 ≤ 7 4 5  → run. Work = 7 4 5 + 3 0 2 = 10 4 7
P0 need 7 4 3 ≤ 10 4 7 → run. Work = 10 5 7
```

Safe sequence: **<P1, P3, P4, P2, P0>** → the state is **safe**. (Other valid sequences may exist; GATE asks "is it safe?" or "which sequence is valid?".)

Minimum resources for deadlock freedom Core

FORMULA / RULE

```
n processes, each needs at most m units of one resource
type.
Deadlock-free if   R ≥ n(m − 1) + 1
```

Logic: worst case each process holds m−1 units and all wait for one more. One extra unit lets someone finish.

WORKED EXAMPLE

Worked numerical

Q: 3 processes each need up to 4 units of the same resource. Minimum units to guarantee no deadlock?
A: R = 3(4−1) + 1 = **10**. With 9 units, each could hold 3 and all be stuck.

WORKED EXAMPLE

Worked numerical

Q: 12 units of a resource, each process needs at most 3. Maximum number of processes that keeps the system deadlock-free?
A: Need 12 ≥ n(3−1)+1 = 2n+1 → n ≤ 5.5 → **5 processes**.

Wait-for graph High-yield

FORMULA / RULE

When every resource has a **single instance**, the resource-allocation graph can be collapsed into a **wait-for graph**: delete the resource nodes and draw **Pi → Pj** whenever Pi is waiting for a resource currently held by Pj. A **cycle in the wait-for graph ⇔ deadlock**. With multiple instances per resource, a cycle in the RAG is only *necessary* — use the detection algorithm instead (see Q12). The OS runs cycle detection periodically, or when CPU utilisation drops; a cycle search on n processes costs O(n²). (DBMS uses the same idea for transactions.)

WORKED EXAMPLE

Worked example

Q: P1 holds R1 and requests R2; P2 holds R2 and requests R1; P3 holds R3 and requests R1. Is there a deadlock?
A: Wait-for edges: P1 → P2 (R2 is held by P2), P2 → P1 (R1 is held by P1), P3 → P1 (R1 is held by P1).

```
P1 ⇄ P2   is a cycle → P1 and P2 are deadlocked.
P3 → P1   is only an edge INTO the cycle: P3 is blocked
behind the deadlock but is not on a cycle.
```

5 Memory Management & Paging

Logical vs physical addresses Core

NOTES

A program uses **logical (virtual) addresses**, as if it owns memory starting at 0. The hardware **MMU** translates them into **physical addresses** in real RAM. This gives protection (you cannot reach another process's memory) and lets the OS place programs anywhere.

Contiguous allocation & fragmentation Core

KEY POINT

Internal fragmentation
Wasted space *inside* an allocated block (fixed partitions, or the last page of a process).

KEY POINT

External fragmentation
Enough total free memory, but split into holes too small to use. Fixed by **compaction** or by paging.

REFERENCE TABLE

Strategy	Picks
First fit	First hole big enough (fast)
Best fit	Smallest hole big enough (leaves tiny leftovers)
Worst fit	Largest hole (leaves a big leftover)

WORKED EXAMPLE

Worked numerical

Q: Free holes (in order): 100K, 500K, 200K, 300K, 600K. Processes arrive: 212K, 417K, 112K, 426K. Place them with First, Best and Worst fit.
A:

```
First fit: 212→500 (rem 288) | 417→600 (rem 183) | 112→288
(rem 176) | 426 → NO hole ≥ 426 → waits
Best fit : 212→300 (rem 88)  | 417→500 (rem 83)  | 112→200
(rem 88)  | 426→600 (rem 174) → all placed
Worst fit: 212→600 (rem 388) | 417→500 (rem 83)  | 112→388
(rem 276) | 426 → NO hole → waits
```

Best fit is the only one that places all four here. No algorithm always wins.

Paging Core

NOTES

Split logical memory into fixed-size **pages** and physical memory into same-size **frames**. Any page can go into any free frame, so there is **no external fragmentation** (only a bit of internal fragmentation in the last page). A **page table** maps page number → frame number.

FLOW

Logical address = [page no | offset] → Page table[page no] = frame no → Physical address = [frame no | offset]

KEY POINT

```
offset bits      = log2(page size)
page-number bits = logical address bits − offset bits
frame-number bits= physical address bits − offset bits
page table size  = (number of pages) × (PTE size)
```

WORKED EXAMPLE

Worked numerical

Q: 32-bit logical address, 4 KB pages, 1 GB physical memory, 4-byte page table entry. Find offset bits, page bits, frame bits and the page table size.
A: Offset = $\log_2 4K$ = **12**. Page bits = 32−12 = **20** → 2^{20} pages. Physical address = 30 bits → frame bits = 30−12 = **18**. Page table = $2^{20} \times 4$ B = **4 MB** per process.

Multilevel paging

NOTES

A 4 MB page table per process is wasteful, so we page the page table itself. With 4 KB pages and 4-byte PTEs, one page holds 4096/4 = 1024 entries = 2^{10}. So the 20 page bits split into **10 (outer) + 10 (inner)**: address = `[10 | 10 | 12]` . Only the parts of the table actually in use need to exist. Cost: more memory accesses on a miss (2 table lookups + 1 data access for 2 levels).

TLB and effective access time

NOTES

The **TLB** is a tiny, fast cache of recent page-table entries. Hit → skip the page table lookup.

KEY POINT

```
EAT = h × (t_tlb + t_mem)  +  (1 − h) × (t_tlb + 2·t_mem)
(single-level page table)
```

For a k-level table, a TLB miss costs t_tlb + (k+1)·t_mem.

WORKED EXAMPLE

Worked numerical

Q: TLB access 20 ns, memory access 100 ns, TLB hit ratio 90%, single-level paging. Find EAT.
A: EAT = 0.9(20+100) + 0.1(20+200) = 108 + 22 = **130 ns**.

Segmentation (quick)

NOTES

Memory is divided into **variable-size logical units** (code, stack, heap). Address = `(segment no, offset)` . The segment table stores **base and limit**; if `offset ≥ limit` the hardware traps. Segmentation matches how programmers think but suffers external fragmentation; paging is invisible to the programmer.

Inverted page table Extension

KEY POINT

A normal page table has one entry per *virtual* page of *each* process. An **inverted page table** has **one entry per physical frame**: each entry records **(process id, page number)** of the page currently in that frame, and the index of the matching entry *is* the frame number. Translation searches the table for (pid, page) — made fast with a **hash table** plus a TLB.

REFERENCE TABLE

	Normal page table	Inverted page table
Entries	Virtual pages, per process	Physical frames, one table for the whole system
Size depends on	Virtual address-space size × number of processes	**RAM size only**
Lookup	Direct index by page number	Search / hash on (pid, page) — slower
Shared pages	Easy (two tables point at one frame)	Hard (one entry holds only one (pid, page))

WORKED EXAMPLE

Worked numerical

Q: 32-bit virtual addresses, 4 KB pages, 4-byte entries, 64 MB of RAM. Compare the page-table size per process with an inverted table using 8-byte entries.
A: Normal: $2^{32}/2^{12} = 2^{20}$ pages × 4 B = **4 MB per process**. Inverted: 64 MB / 4 KB = 2^{14} = 16384 frames × 8 B = **128 KB for the whole system**, however many processes exist.

Buddy system Extension

KEY POINT

Memory is allocated in blocks whose size is a **power of two**. A request is rounded **up** to the next power of two; if no block of that size is free, a larger block is repeatedly **split in half** — the two halves are **buddies**. On release, a block is **merged with its buddy** if that buddy is also free, and the merge repeats upward. A block's buddy address = **block address XOR block size**. Merging is fast, but the rounding causes **internal fragmentation** (up to nearly 50%). Linux uses it for physical page frames.

WORKED EXAMPLE

Worked example

Q: From 1024 KB, allocate a 70 KB request.
A:

```
1024 → split into 512 + 512      (use one 512)
 512 → split into 256 + 256
 256 → split into 128 + 128      128 ≥ 70 and 64 < 70  →
allocate a 128 KB block
Free blocks left: 512 (other half), 256, 128.   Internal
fragmentation = 128 − 70 = 58 KB.
```

6 Virtual Memory

Demand paging & page faults Core

NOTES

A page is loaded only when first used. If the page table says the page is not in memory (valid bit = 0), the CPU raises a **page fault**:

FLOW

Trap to OS → Find page on disk → Get a free frame (or evict one) → Read page in, update table → Restart the instruction

KEY POINT

```
EAT = (1 − p) × t_mem  +  p × t_fault        (p = page fault
rate)
```

WORKED EXAMPLE

Worked numerical

Q: Memory access 200 ns, page fault service time 8 ms, p = 1/1000. Find EAT, then the maximum p for at most 10% slowdown.
A: t_fault = 8,000,000 ns. EAT = 0.999×200 + 0.001×8,000,000 = 199.8 + 8000 ≈ **8200 ns** (a 40× slowdown!).
For ≤10% slowdown: EAT ≤ 220 → 200 + p(8,000,000 − 200) ≤ 220 → p ≤ 20 / 7,999,800 ≈ **2.5×10^{-6}** (about 1 fault per 400,000 accesses). Page faults must be extremely rare.

Page replacement algorithms Core

NOTES

When memory is full, pick a victim page to evict.

REFERENCE TABLE

Algorithm	Evicts	Notes
FIFO	Oldest loaded page	Simple; suffers **Belady's anomaly**
Optimal (OPT)	Page not needed for the longest time in future	Fewest faults, but needs the future (used as a benchmark)
LRU	Least recently used page	Good approximation of OPT; needs hardware support

WORKED EXAMPLE

Worked numerical: Belady's anomaly

Q: Reference string 1,2,3,4,1,2,5,1,2,3,4,5 with FIFO. Count faults with 3 frames and with 4 frames.
A:

```
3 frames:  1F 2F 3F  4F(evict 1) 1F(evict 2) 2F(evict 3)
5F(evict 4) 1H 2H 3F(evict 1) 4F(evict 2) 5H
           faults = 9
4 frames:  1F 2F 3F 4F  1H 2H  5F(evict 1) 1F(evict 2)
2F(evict 3) 3F(evict 4) 4F(evict 5) 5F(evict 1)
           faults = 10
```

More frames gave **more** faults: that is Belady's anomaly. It happens with FIFO but never with LRU or OPT (these are **stack algorithms**: the pages held with k frames are always a subset of those held with k+1).

COMMON TRAP

Classic result

For the textbook string 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 with 3 frames: **FIFO = 15, LRU = 12, OPT = 9** faults. Always expect OPT ≤ LRU ≤ FIFO for such strings (not guaranteed for FIFO in every case, but typical).

Thrashing Core

NOTES

If a process does not have enough frames, it faults constantly and spends more time swapping than computing. CPU utilization **drops**, and a naive OS may then admit *more* processes, making it worse. Cure: reduce the degree of multiprogramming or use the **working-set model** (give each process enough frames for the pages it actively uses).

Second-chance (Clock) page replacement High-yield

KEY POINT

FIFO with a safety valve. Frames sit in a **circular list** with a pointer (the clock **hand**); each frame has a **reference bit** that is set to 1 when the page is used. On a fault the hand sweeps: if the bit is **1** it is cleared to 0 and the hand moves on (a *second chance*); if it is **0** that page is the victim. The new page is loaded with its bit = 1 and the hand moves past it. If every bit is 1 the hand clears them all and replaces the first frame — **the algorithm degenerates to FIFO**.

Enhanced clock uses the pair (reference, modify): (0,0) best victim → (0,1) → (1,0) → (1,1) worst, because clean, unused pages are cheapest to evict. A full trace is in solved problem Q16.

LFU, MFU and counting-based replacement Extension

REFERENCE TABLE

Policy	Victim	Reasoning and weakness
LFU — least frequently used	Page with the **smallest** reference count	Actively used pages have large counts. Weakness: a page used heavily early on keeps its big count forever (fix: periodically halve the counts)
MFU — most frequently used	Page with the **largest** count	The page with the smallest count was probably just brought in and has not been used yet. Rare in practice

NOTES

Neither is common; real systems use LRU approximations (reference bit, additional-reference-bits **aging**, second chance). Recall that FIFO suffers Belady's anomaly while LRU and OPT (stack algorithms) do not.

Frame allocation & global vs local replacement High-yield

REFERENCE TABLE

Scheme	Frames given to process i
Equal	m / n (m frames, n processes)
Proportional	$a_i = (s_i / S) \times m$, where s_i = size of process i and $S = \Sigma\, s_i$
Priority	Proportional to priority (or to a mix of priority and size)

NOTES

Every process needs a **minimum** number of frames, fixed by the instruction set (the most pages one instruction can touch, e.g. with indirect addressing).

KEY POINT

Global replacement

The victim may be a frame of *any* process. Higher throughput and flexible, but a process's fault rate depends on the others, and a high-priority process can steal frames — thrashing can spread.

KEY POINT

Local replacement

The victim must come from the process's *own* frames. Performance is predictable, but free frames may sit idle while a process thrashes within its fixed allocation.

WORKED EXAMPLE

Worked numerical

Q: 62 free frames are shared between processes of 10 pages and 127 pages using proportional allocation (round down).
A: S = 137. $a_1 = 62 \times 10/137 = 4.53 \rightarrow$ **4**; $a_2 = 62 \times 127/137 = 57.47 \rightarrow$ **57**. (One frame is left over after rounding down.)

7 File Systems, Disk Scheduling & I/O

File allocation methods Core

REFERENCE TABLE

Method	How	Pros / Cons
Contiguous	File occupies consecutive blocks	Fast sequential/direct access; external fragmentation, hard to grow
Linked	Each block points to the next	No fragmentation; slow random access, pointer loss breaks the chain (FAT keeps the links in a table)
Indexed	An index block lists all data blocks	Fast direct access; index block overhead

Unix inode

NOTES

The inode holds file attributes plus block pointers: some **direct**, then **single**, **double** and **triple indirect** pointers. Small files are reached fast, huge files are still possible.

WORKED EXAMPLE

Worked numerical

Q: Inode with 10 direct pointers and 1 single, 1 double, 1 triple indirect pointer. Block size 1 KB, block pointer 4 bytes. Maximum file size?
A: Pointers per block = 1024/4 = 256.

```
Direct          = 10 blocks
Single indirect = 256
Double indirect = 256² = 65,536
Triple indirect = 256³ = 16,777,216
Total blocks    = 16,843,018  →  ≈ 16,843,018 KB ≈ 16 GB
```

Almost all of the capacity comes from the triple-indirect level.

Disk access time Core

KEY POINT

```
Access time = seek time + rotational latency + transfer time
Average rotational latency = ½ × (time for one rotation)
```

WORKED EXAMPLE

Worked numerical

Q: 7200 RPM disk, average seek 9 ms, transfer rate 100 MB/s. Time to read a 4 KB block?
A: One rotation = 60/7200 s = 8.33 ms → average latency = **4.17 ms**. Transfer = 4 KB / 100 MB/s ≈ **0.04 ms**. Total ≈ 9 + 4.17 + 0.04 ≈ **13.2 ms**. Seek and rotation dominate, so scheduling to reduce head movement matters.

Disk scheduling Core

REFERENCE TABLE

Algorithm	Idea
FCFS	Serve in arrival order (fair, lots of movement)
SSTF	Nearest request first (can starve far requests)
SCAN (elevator)	Sweep to one end of the disk serving requests, then reverse
C-SCAN	Sweep one way only; at the end jump back to the start (more uniform wait)
LOOK / C-LOOK	Like SCAN / C-SCAN but turn around at the last request instead of the disk end

WORKED EXAMPLE

Worked numerical

Q: Disk cylinders 0–199, head at 53 moving toward 0. Request queue: 98, 183, 37, 122, 14, 124, 65, 67. Find total head movement for each algorithm.
A:

```
FCFS: 53→98→183→37→122→14→124→65→67
      45+85+146+85+108+110+59+2 = 640

SSTF: 53→65→67→37→14→98→122→124→183
      12+2+30+23+84+24+2+59 = 236

SCAN (going down first): 53→37→14→0, then reverse
0→65→67→98→122→124→183
      53 + 183 = 236

LOOK (going down first): 53→14 (39), then 14→183 (169)
      39 + 169 = 208
```

LOOK < SCAN because it does not travel to cylinder 0. When a question says "C-SCAN", check whether the return jump is counted in the total, and follow what the question states.

Free-space management Core

REFERENCE TABLE

Method	How it works	Trade-off
Bit vector (bitmap)	1 bit per block: 1 = free (or 0 = free, by convention)	Simple; finding contiguous runs is easy; the map takes **blocks / 8 bytes** of space
Linked list	Free blocks chained through a pointer in each block	No wasted map, but traversing the list is slow and contiguous runs are hard to find
Grouping	First free block holds the addresses of n free blocks; the last of those points to the next such block	Many free blocks found quickly
Counting	Store (start block, run length) pairs	Compact when free space is in long contiguous runs

WORKED EXAMPLE

Worked numerical

Q: Bit-vector size for a 512 GB disk with 4 KB blocks?
A: Blocks = 2^{39} / 2^{12} = 2^{27}. One bit each → 2^{27} bits = 2^{24} bytes = **16 MB**.

Directory structures Core

REFERENCE TABLE

Structure	Key property	Problem
Single-level	One directory for all files	Name clashes between users; no grouping
Two-level	A master directory with one directory per user	No sharing between users; no sub-grouping
Tree	Arbitrary depth; absolute and relative path names	Sharing is awkward; deleting a non-empty directory needs a policy
Acyclic graph	Shared files and subdirectories via links	Dangling pointers and several names per file; use reference counts
General graph	Cycles allowed	Traversal can loop forever; needs cycle detection or garbage collection

Hard links vs soft (symbolic) links High-yield

REFERENCE TABLE

	Hard link	Soft link
What it is	Another **directory entry** pointing at the **same inode**	A **separate file** (own inode) whose content is a path name
Link count	Stored in the inode; data are freed only when it reaches 0	Does not change the target's link count
Crosses file systems?	No (inode numbers are per file system)	Yes
Can link a directory?	Normally not (would create cycles)	Yes
If the original name is deleted	Data still reachable through the other name	Becomes **dangling** (broken)

WORKED EXAMPLE

Worked example

Q: File `a` has link count 1. After `ln a b` (hard) and `ln -s a c` (soft), then `rm a`, what happens?
A: `ln a b` raises the count to 2; the soft link `c` does not change it. `rm a` drops the count to 1, so the data stay reachable through `b`. But `c` stores the path "a", which no longer exists, so `c` is **dangling**.

FAT file system High-yield

KEY POINT

The **File Allocation Table** sits at the start of the volume and has **one entry per disk block (cluster)**. A directory entry stores only the **first block number** of a file; each FAT entry holds the **next block** of the chain, an **end-of-file marker**, or a free/bad marker. Entry widths give FAT-12, FAT-16 and FAT-32. Reaching the k-th block means following k − 1 links — cheap when the FAT is cached in memory, because no data block has to be read to find the chain. **FAT size = number of blocks × entry size.**

WORKED EXAMPLE

Worked numerical

Q: 512 GB disk, 4 KB blocks, 4-byte FAT entries. FAT size? How many FAT entries does a 100 KB file use?
A: Blocks = 2^{27}, so FAT = $2^{27} \times 4$ B = 2^{29} B = **512 MB**. The file occupies 100/4 = **25 blocks** → a chain of 25 FAT entries.

I/O subsystem: polling, interrupts and DMA High-yield

REFERENCE TABLE

Technique	How data move	CPU cost
Programmed I/O (polling)	CPU repeatedly reads the device status register (busy-waiting), then copies each word	Highest — the CPU is tied up for the whole transfer
Interrupt-driven I/O	Device interrupts when ready; the CPU copies each byte or word	One interrupt per byte or word
DMA	A DMA controller moves a whole block between device and memory without the CPU	Setup plus **one interrupt per block**

KEY POINT

DMA steps: (1) the CPU programs the controller with the memory address, byte count and direction; (2) the controller requests the bus; (3) it transfers the block directly; (4) it raises **one interrupt** when the count reaches zero.

Modes: burst (block) mode — the controller keeps the bus for the entire block, so the CPU is locked out of memory meanwhile; **cycle stealing** — it takes the bus one word at a time between CPU cycles, slowing the CPU slightly; **transparent mode** — it transfers only when the CPU is not using the bus.

WORKED EXAMPLE

Worked numerical

Q: A 1 GHz CPU serves a device moving 100 blocks of 16 KB per second. Interrupt-driven I/O costs 300 cycles per byte; DMA costs 1500 cycles per block (setup + completion interrupt). What fraction of CPU time goes to I/O in each case?
A: Interrupt-driven: 16384 × 300 = 4,915,200 cycles per block × 100 = 4.9152 × 10^8 cycles/s → **49.15%** of the CPU. DMA: 1500 × 100 = 1.5 × 10^5 cycles/s → **0.015%**.

Buffering & spooling High-yield

REFERENCE TABLE

Concept	Purpose
Buffering (single, double, circular)	Hides speed mismatch between producer and consumer, adapts different transfer sizes, and supports "copy semantics". With **double buffering** one buffer is filled while the other is drained, so the two overlap
Cache vs buffer	A cache holds a *copy* of data that also exists elsewhere (for speed); a buffer may hold the *only* copy of data in transit
Spooling	Output for a device that cannot interleave jobs (a printer) is first written to a disk file; a spooler daemon then sends the files to the device **one job at a time**. This stops jobs interleaving and lets the CPU carry on while the slow device works

RAID (redundant arrays of disks) Extension

NOTES

RAID combines several disks for **speed** (striping) and/or **fault tolerance** (mirroring or parity). It is a lighter GATE topic than disk scheduling, but the capacity and failure rules are quick marks.

REFERENCE TABLE

Level	Idea	Usable capacity (N disks of size D)	Failures survived
RAID 0	Striping, no redundancy	N × D	**0** — one failure loses everything
RAID 1	Mirroring (every block written to two disks)	N × D / 2	1 per mirrored pair (at least 1)
RAID 5	Block striping with **distributed parity** (parity = XOR of the data blocks of a stripe)	(N − 1) × D	**1**
RAID 6	Two independent parity blocks per stripe	(N − 2) × D	**2**
RAID 10	Stripe across mirrored pairs	N × D / 2	1 per pair

KEY POINT

Parity recovery. If a stripe holds blocks A, B, C and parity P = A ⊕ B ⊕ C, a lost block is rebuilt by XOR-ing all the survivors: e.g. A = B ⊕ C ⊕ P.

RAID 5 small-write penalty. Updating one data block needs **4 disk I/Os**: read old data, read old parity, write new data, write new parity (new P = old P ⊕ old data ⊕ new data). Because parity is **distributed** across all disks rather than kept on one dedicated disk (RAID 4), no single parity disk becomes the write bottleneck.

WORKED EXAMPLE

Worked numerical

Q: Six 4 TB disks are used as (a) RAID 0, (b) RAID 1, (c) RAID 5, (d) RAID 6. Find the usable capacity of each.
A: (a) 6 × 4 = **24 TB**; (b) 24/2 = **12 TB**; (c) (6 − 1) × 4 = **20 TB**; (d) (6 − 2) × 4 = **16 TB**.

8 Problem → Standard Method Playbook

METHOD

1. Scheduling

Build the execution timeline first. Mark arrivals and preemptions, then calculate completion time `C`, turnaround `C−A`, waiting `turnaround−burst`, and response `first start−arrival`. Add context-switch overhead only where the schedule actually switches.

METHOD

2. HRRN / MLFQ

HRRN is non-preemptive: at each completion choose the arrived job maximizing `(waiting+burst)/burst`. MLFQ is priority-queue based: determine the queue/quantum rule, then apply demotion, promotion/aging, and preemption exactly as specified.

METHOD

3. Deadlock / Banker

Separate *prevention*, *avoidance*, and *detection*. For Banker, compute `Need = Max−Allocation`, maintain `Work`, and repeatedly find a process whose Need ≤ Work; then release its Allocation. A complete safe sequence proves the state is safe.

METHOD

4. Paging / TLB

Split address into page number + offset. Offset bits = $\log_2(\text{page size})$. For EAT, enumerate TLB-hit and TLB-miss paths rather than memorising a formula blindly.

METHOD

5. Page replacement

Maintain the frame contents after every reference. FIFO → oldest arrival; LRU → least recently referenced; OPT → page whose next use is farthest in the future; Clock → reference-bit second chance. Count faults only after checking whether the referenced page is already resident.

METHOD

6. Disk scheduling

Draw the cylinder line and mark the initial head. For SCAN/C-SCAN, determine direction and whether the endpoint is physically visited under the question's convention. Total movement is the sum of absolute head-position differences.

METHOD

7. File systems / links

Distinguish inode metadata, directory entries, allocation method, and free-space management. Hard links share an inode; symbolic links store a pathname and can dangle. In FAT, follow the FAT chain from the first block.

METHOD

8. I/O

Polling = CPU repeatedly checks status; interrupt-driven I/O = device notifies CPU; DMA = controller transfers a block with minimal CPU intervention. Buffering handles rate mismatch; spooling queues whole jobs for a dedicated device.

9 🎯 Representative GATE-Style Worked Problems

NOTE

HRRN

Q: At time 0, P1 arrives with burst 8. At time 2, P2 arrives with burst 4; at time 4, P3 arrives with burst 2. Which process runs after P1 under HRRN?
A: P1 finishes at t=8. Waiting: P2=6, P3=4. Ratios: P2=(6+4)/4=**2.5**; P3=(4+2)/2=**3**. Therefore **P3** runs next.

NOTE

MLFQ

Q: A job uses its entire quantum repeatedly while another frequently blocks for I/O. Under the standard MLFQ rule that full-quantum jobs are demoted, what happens?
A: The CPU-bound job moves to a lower-priority queue; the I/O-bound job tends to remain in a higher queue. Aging/priority boost can later prevent starvation.

NOTE

Banker's algorithm

Q: A system has Available = (3,3,2). P1 Need=(1,2,2), P2 Need=(2,1,0), P3 Need=(4,0,1). Which process can safely finish first?
A: Compare Need with Available component-wise. P1 satisfies (1,2,2) ≤ (3,3,2), so P1 can finish and release its Allocation. P2 also satisfies the current Work vector. A complete safe sequence must then be constructed; one qualifying first choice alone is not sufficient to claim the entire state safe.

NOTE

Paging

Q: A 32-bit address uses 4 KB pages. How many bits are the page offset?
A: 4 KB = 2^{12} bytes, so the offset is **12 bits**. The remaining **20 bits** identify the virtual page.

NOTE

LRU

Q: With 3 frames and reference string 1,2,3,1,4,2,1,3,4, how is an LRU replacement decided at the reference to 4?
A: Before 4, frames contain 1,2,3. Their most recent uses are 1 at the immediately previous reference, 2 at the earlier reference, and 3 at the initial reference. Therefore **3** is least recently used and is replaced.

NOTE

Disk scheduling

Q: The disk head is at 50 and requests are 82, 170, 43, 140. Under FCFS, what total head movement occurs?
A: Movement = |50−82| + |82−170| + |170−43| + |43−140| = 32+88+127+97 = **344 cylinders**.

10 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- **Banker:** Need = Max−Allocation; a safe sequence repeatedly selects Need ≤ Work and then adds Allocation to Work.
- Disk head movement is the sum of absolute differences between consecutive head positions.
- FCFS can suffer convoy effect; SJF/SRTF minimise average waiting time when burst information is known under standard assumptions.
- Page-table address translation splits a virtual address into page number + offset; offset bits depend on page size.
- HRRN response ratio = (waiting + service) / service; it is non-preemptive and picks the highest ratio, so long-waiting jobs cannot starve.
- **Buddy system:** round each request up to a power of two (internal fragmentation); buddy address = block address XOR block size.
- Proportional frame allocation $a_i = (s_i / S) \times m$; bit-vector size = blocks / 8 bytes; FAT size = blocks × entry size.
- **RAID usable capacity:** RAID 0 = N·D, RAID 1/10 = N·D/2, RAID 5 = (N−1)·D, RAID 6 = (N−2)·D; RAID 5 small write = 4 I/Os.

LAST-MINUTE · MUST REMEMBER

- MLFQ is defined by its queue/quantum/promotion rules; always simulate the specified policy rather than assuming one universal implementation.
- FIFO can show Belady's anomaly; LRU and OPT are stack algorithms and do not.
- A process has its own address space/resources; threads within a process share the process address space and many resources.
- Context switching changes execution context but does not itself perform useful application work.
- Race conditions arise when concurrent execution accesses shared state and correctness depends on timing/interleaving.
- A critical section requires mutual exclusion plus appropriate progress/bounded-waiting properties for the intended solution.
- **Deadlock requires all four:** mutual exclusion, hold-and-wait, no preemption, and circular wait.
- TLB is a cache of page-table translations; a TLB miss does not necessarily mean a page fault.
- Belady's anomaly can occur in FIFO page replacement; it does not occur in stack algorithms such as LRU and OPT.
- In a file system, distinguish logical file structure, allocation method, directory structure, and free-space management.
- MLFQ moves jobs between queues by observed behaviour: full-quantum users are demoted, I/O-bound jobs stay high, and aging promotes starving jobs.
- DMA interrupts the CPU once per block, not once per byte; burst mode holds the bus for the whole block, cycle stealing takes it one word at a time.
- Test-and-set / compare-and-swap give mutual exclusion and progress but not bounded waiting; they spin (busy-wait), so they suit short critical sections on multiprocessors.

LAST-MINUTE · COMMON TRAPS

- TLB miss ≠ page fault; a TLB miss can still find a valid page-table entry in memory.
- Preemptive and non-preemptive scheduling are different; always check whether a running process can be interrupted.
- Page fault and TLB miss are different events; a TLB miss can still be satisfied from memory-resident page-table entries.
- Zombie = terminated but not yet reaped by wait(); orphan = still running but its parent has died (adopted by init).
- An inverted page table has one entry per physical frame, so its size depends only on RAM, not on the number of processes; lookup needs hashing.
- Hard links share one inode and a link count; soft links are separate files holding a path and can dangle.
- A monitor condition-variable signal is lost if nobody is waiting, unlike a semaphore signal, which is remembered. Under Mesa semantics re-check the condition in a while loop.
- Deadlock = blocked forever in a cycle; livelock = running but making no progress; starvation = never chosen, fixed by aging.

LAST-MINUTE · GATE FAVOURITES

- HRRN chooses the maximum (waiting+burst)/burst at each scheduling decision; it is non-preemptive.
- Banker's algorithm is an avoidance algorithm; deadlock detection is different from prevention and avoidance.
- Disk scheduling questions require careful calculation of head movement; know FCFS, SSTF, SCAN, C-SCAN and their variants.
- **Priority inversion:** a high-priority task waits for a lock held by a low-priority task that a medium-priority task keeps preempting. Fix: priority inheritance.